



HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK



Budapest,
2026.04.22.

SZÁMÍTÓGÉP ARCHITEKTÚRÁK

Pipeline utasításfeldolgozás

Horváth Gábor, Belső Zoltán

BME Hálózati Rendszerek és Szolgáltatások Tanszék
ghorvath@hit.bme.hu, belso@hit.bme.hu

- Most megtanuljuk, hogy lehet hatékonyan.
- Példa processzor: RISC.
- Utasításai:
 - Load/Store
 $R1 \leftarrow \text{MEM}[R0+42]$ vagy $\text{MEM}[R0+42] \leftarrow R1$
 - Aritmetikai logikai
 $R1 \leftarrow R2 * R3$ vagy $R1 \leftarrow 42 * R3$.
 - Vezérlésátadás:
 - Feltétel nélküli
 - Feltételes
 $\text{JUMP } -24 \text{ IF } R2 == 0$
 - Load/Store kivételével más nem nyúlhat memóriához

- Utasítás végrehajtásának lépései:
 - 1) Lehívás (Instruction Fetch, **IF**)
+ utasításszámláló növelése (ha nem ugrás)
 - 2) Dekódolás, regiszterek kiolvasása
(Instruction decode/register fetch, **ID**)
 - Binárisan kódolt utasítás felkoncolása:
 - Típus leválasztása
→ ALU vezérlő jelek
 - Regiszter operandusok kiolvasása a regiszter tárolóból
 - 3) Végrehajtás (Execution, **EX**) – ALU dolgozik
 - Load/Store: címet számol (**R1** $\leftarrow$ **MEM[R0+42]**)
 - Aritmetikai/logikai: eredményt számol (**R1** $\leftarrow$ **R2*R3**)
 - Feltételes ugrás: feltételt és ugrási címet számol
(**JUMP [PC]-24 IF R2==0**)

- ... folytatás:

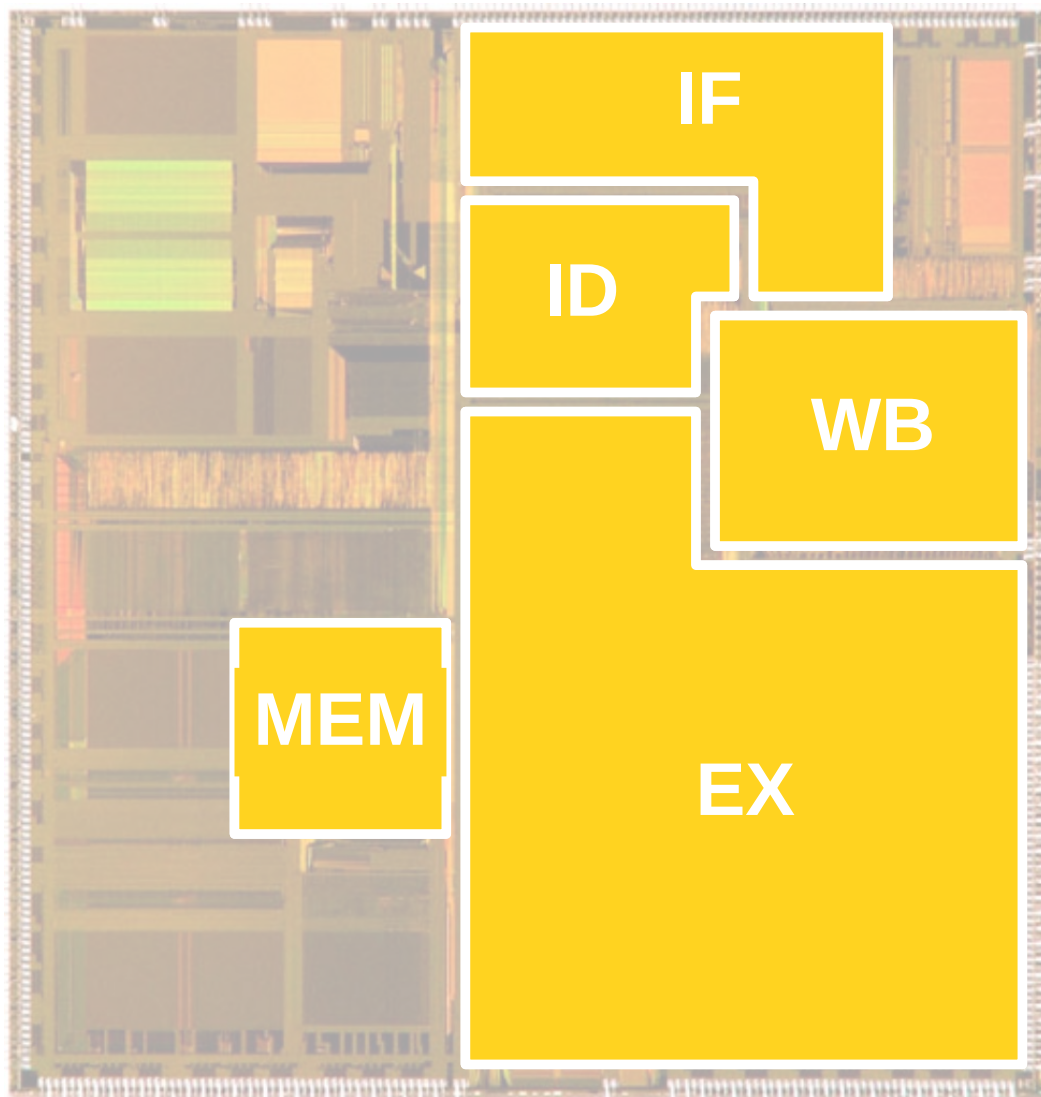
4) Memóriaműveletek (Memory access, **MEM**)

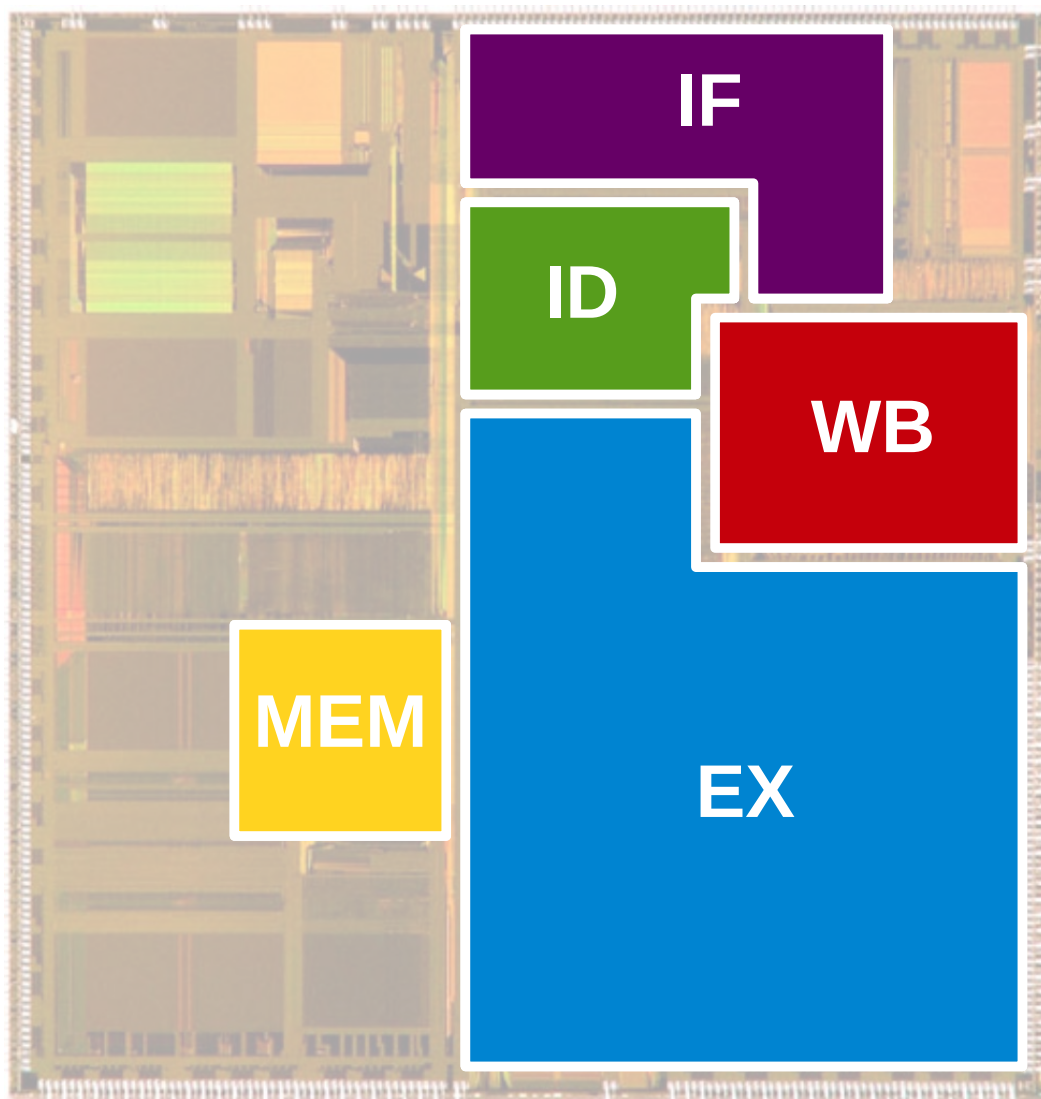
- Load/Store: végrehajtódik
- Aritmetikai/logikai, ugrások: kihagyható

5) Regiszterek frissítése (Write-back, **WB**)

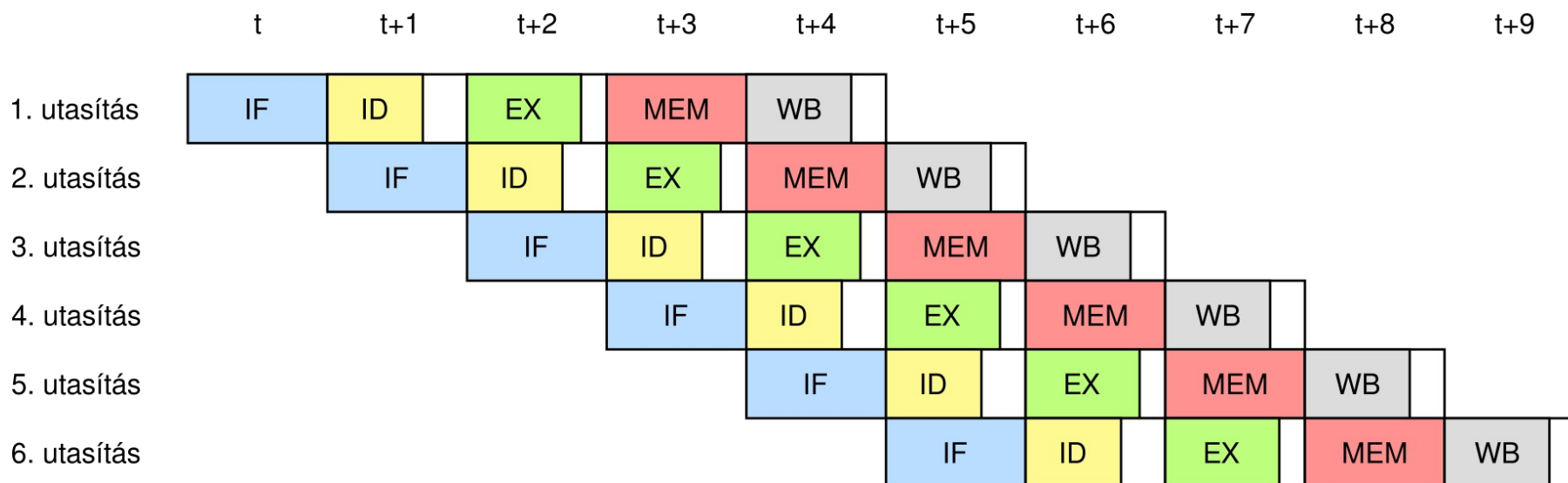
- Aritmetikai/logikai: eredményt tárol (**R1** $\leftarrow$ **R2*****R3**)
- Load: eredményt tárol (**R1** $\leftarrow$ **MEM**[**R0**+42])
- Store, ugrások: kihagyható

- Aritmetikai: IF, ID, EX, WB
- Store: IF, ID, EX, MEM
- Load: IF, ID, EX, MEM, WB
- Ugrások: IF, ID, EX



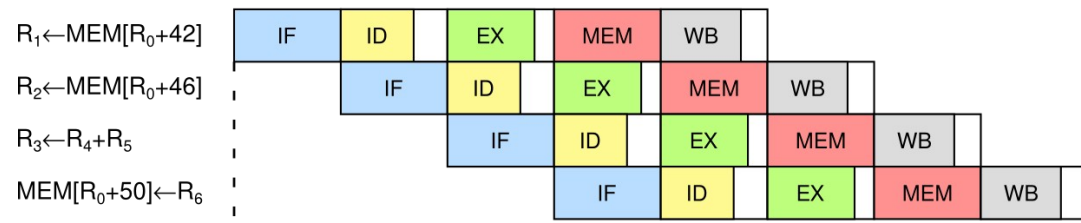


- Ez a pipeline elv
 - Lásd: chicagói vágóhidak, Ford T-model, stb.
- Átlapolt utasításfeldolgozás:
 - Minden fázisnak ugyanannyi időt kell adni: **ciklusidő**

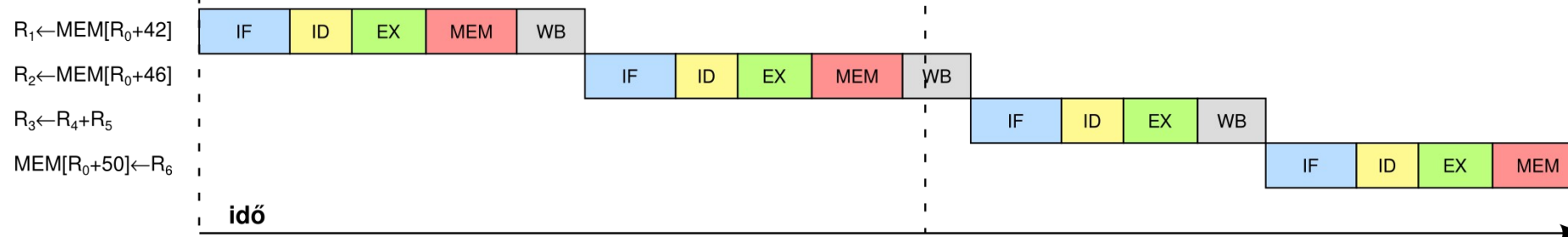


- Mennyit nyerünk?
 - Egyfelől: átlapoltunk, ez nyereség
 - Másfelől: üresjáratok! (szükségtelen fázisok, túl hosszú fázisok, ...)
- Ha függetlenek az utasítások, akkor sokat!

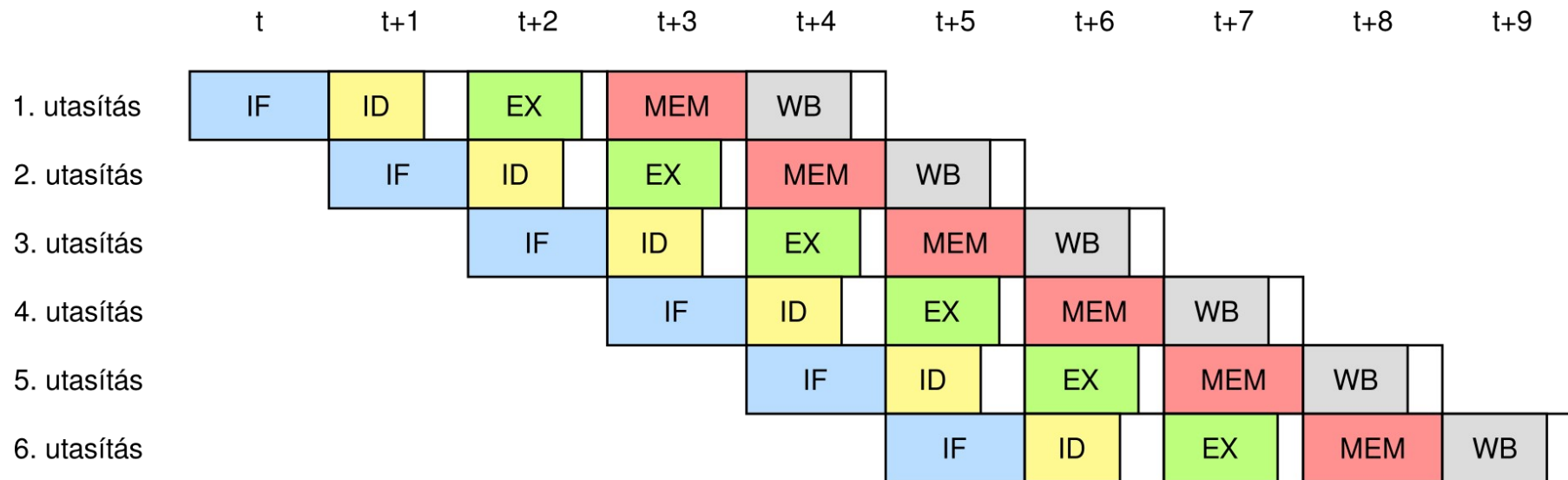
Pipeline-al:



Pipeline nélkül:

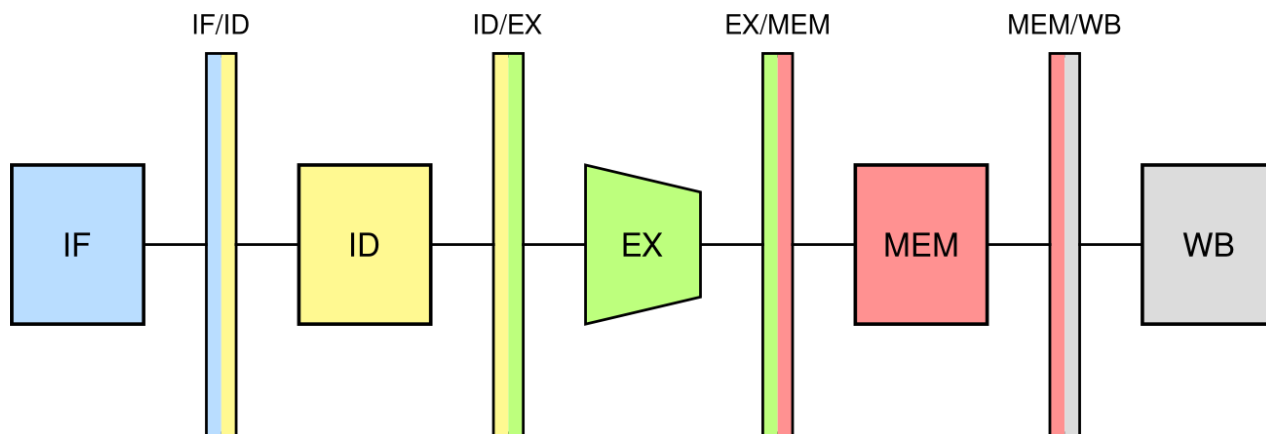


- Mennyiségek:
 - Mélység (deepness): fázisok száma
 - Késleltetés (latency): átfutási idő
 - Átviteli sebesség (throughput): kész utasítás / sec
 - Feltöltési / kiürülési idő



- A fázisok egymásnak adogatják a „munkadarabbal” kapcsolatos dolgokat

→ pipeline regiszterekben tárolódnak



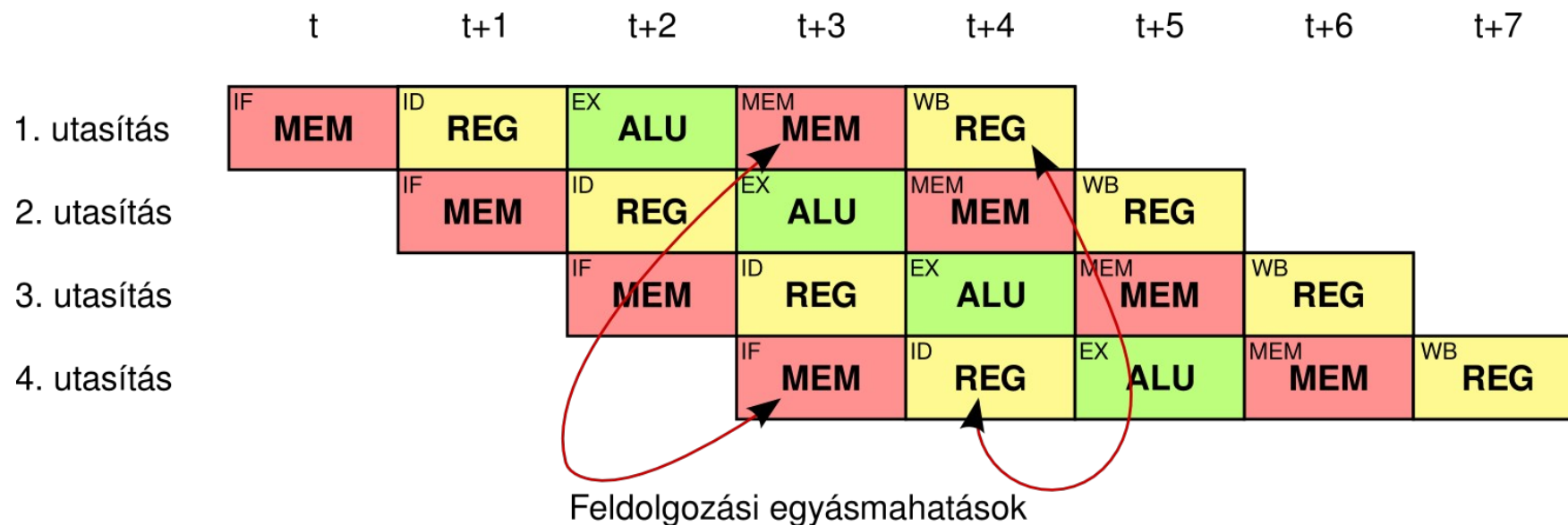
- IF az IF/ID-be teszi a lehívott utasítást,
- ID kiveszi, ALU vezérlőjeleket és operandusokat az ID/EX-be teszi,
- EX kiveszi, eredményt az EX/MEM-be teszi,
- Stb... (később)

- Minden szép és jó, de:
- Az utasítások többnyire nem függetlenek!
 - Erőforrásokért versengenek
 - **Feldolgozási egymásrahatás**
 - Egyik operandusa a másik eredménye
 - **Adat egymásrahatás**
 - Ugrás: nem tudjuk a folytatást, amíg ki nem értékeljük a feltételt
 - **Procedurális egymásrahatás**

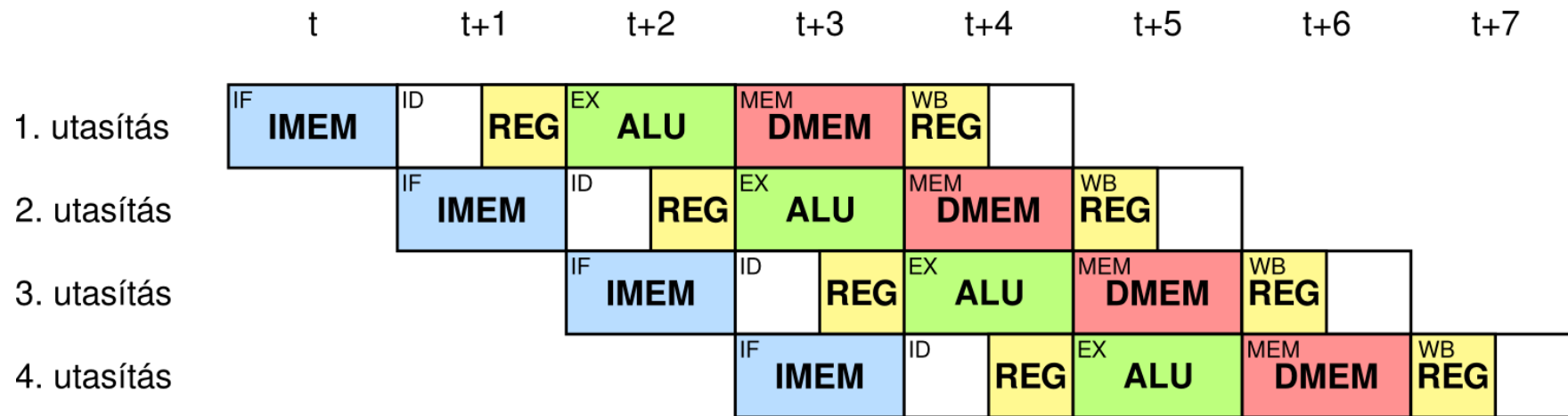
Kezelés:

- Trükkkel feloldjuk
- Ha nem megy, megállítjuk a pipeline-t, amíg meg nem oldódik
 - rontja a hatékonyságot

- Egyes erőforrásokra több fázisban is szükség van
 - IF: **Memória**
 - ID: **Regiszter tároló**
 - EX: ALU
 - MEM: **Memória**
 - WB: **Regiszter tároló**



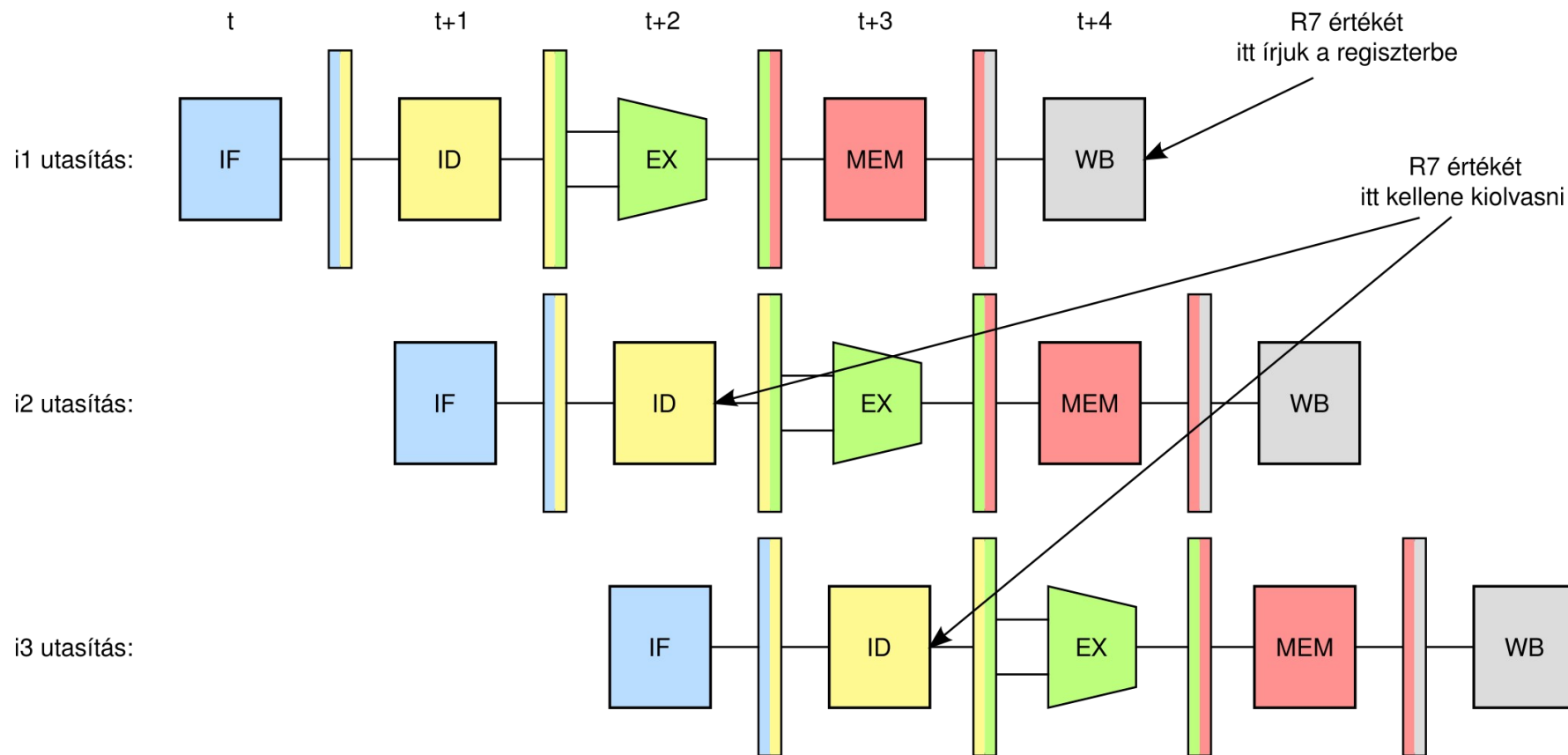
- Feloldása:
 - Memória esetén:
 - külön utasítás és adat cache
 - Regiszter tároló esetén:
 - egyik az ciklus elején, másik a végén használhatja



- Forrása: adatfüggőségek
- Adatfüggőség:
 - Több utasítás ugyanazon a címen végez műveletet**
(ugyanaz a regiszter, ugyanaz a memóriacím)
- Pl. az egyik utasítás operandusa az előző eredménye
→ **RAW függőség** (Read After Write)

```
i1:  R3 ← MEM[R2]
i2:  R1 ← R2 * R3
i3:  R4 ← R1 + R5
i4:  R5 ← R6 + R7
i5:  R1 ← R8 + R9
```

RAW függőségek: $i1 \leftrightarrow i2$, $i2 \leftrightarrow i3$

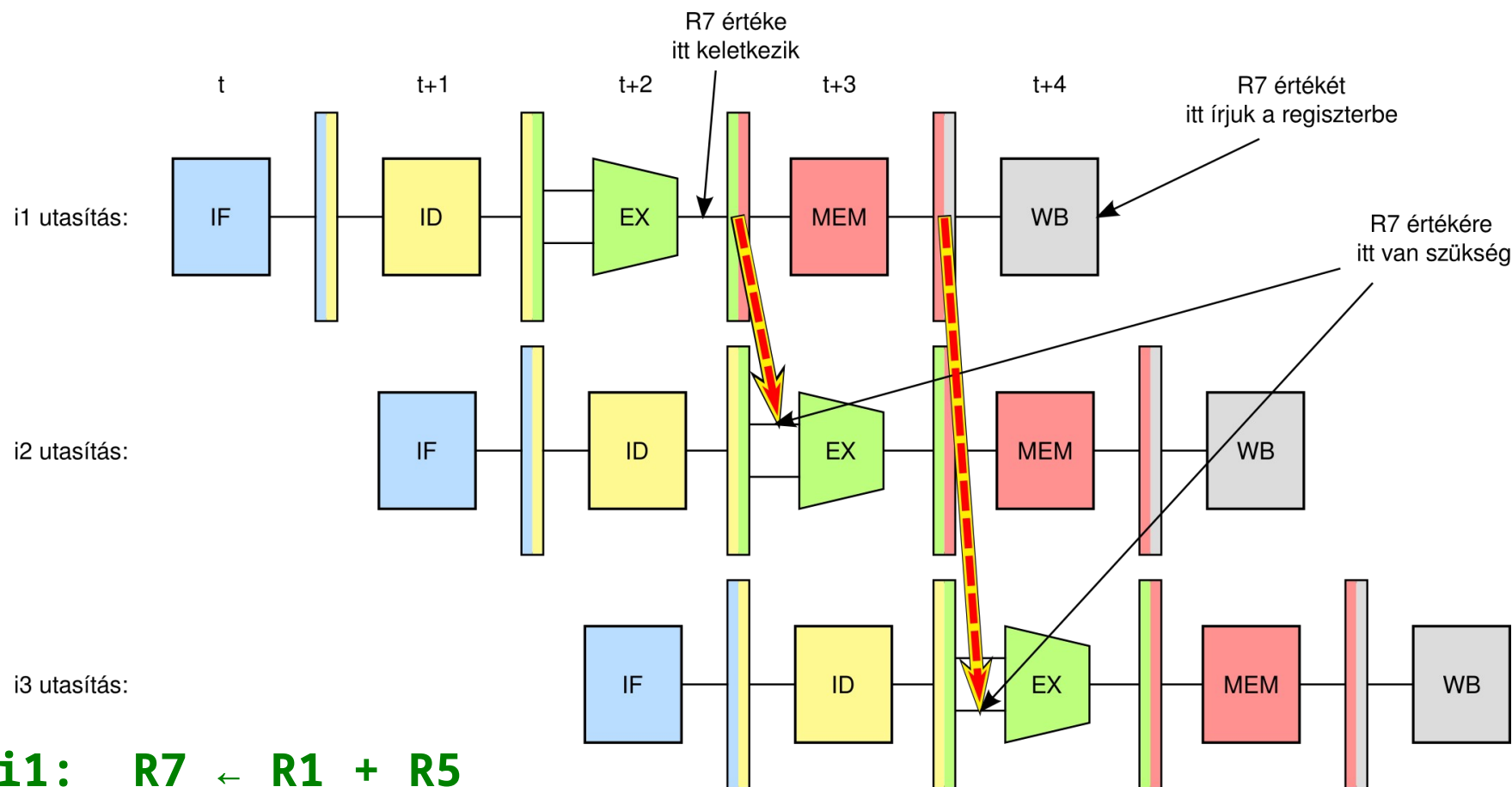


i1: R7 ← R1 + R5

i2: R8 ← R7 + R2

i3: R5 ← R8 + R7

- Kész van az az eredmény, csak rossz helyen → **forwarding**



i1: R7 ← R1 + R5

i2: R8 ← R7 + R2

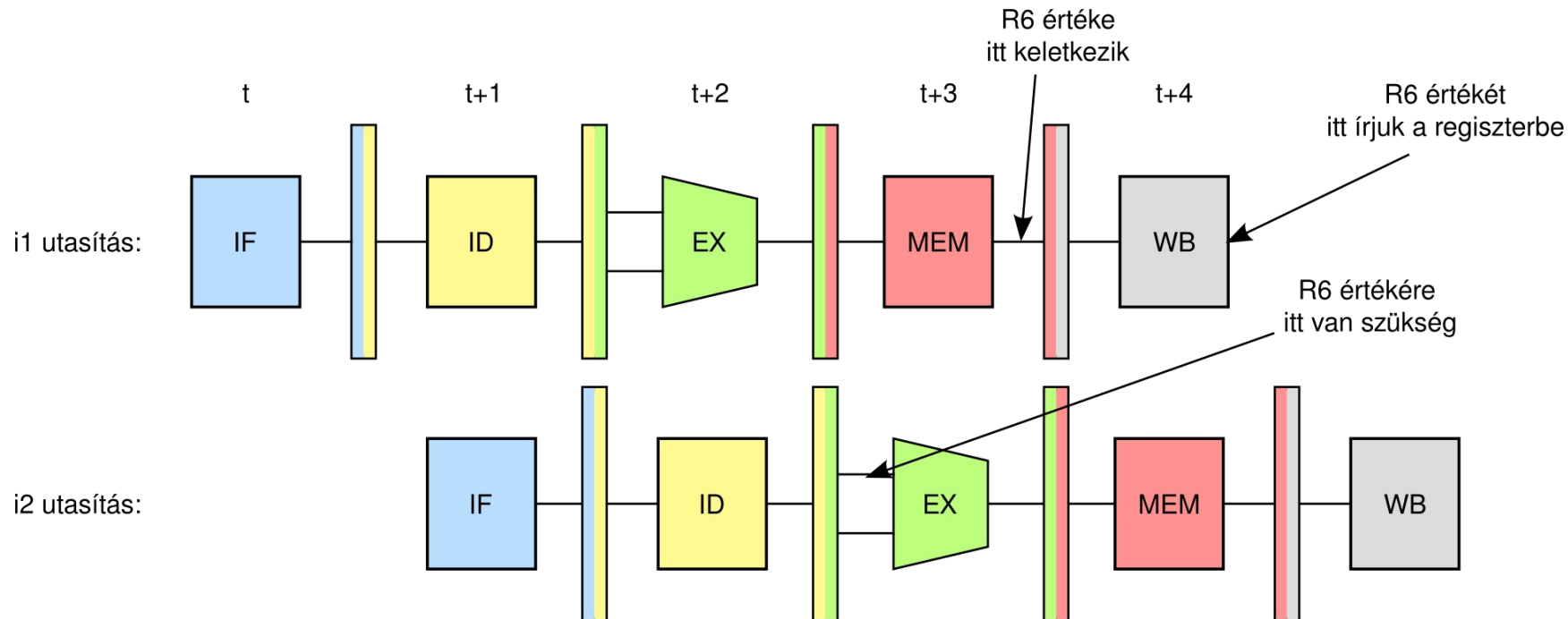
i3: R5 ← R8 + R7

- Forwarding: van, ami ellen nem véd:

i1: R6 ← MEM[R2]

i2: R7 ← R6 + R4

- R6: értéke a MEM végén áll elő
- i2-nek az EX elején kellene

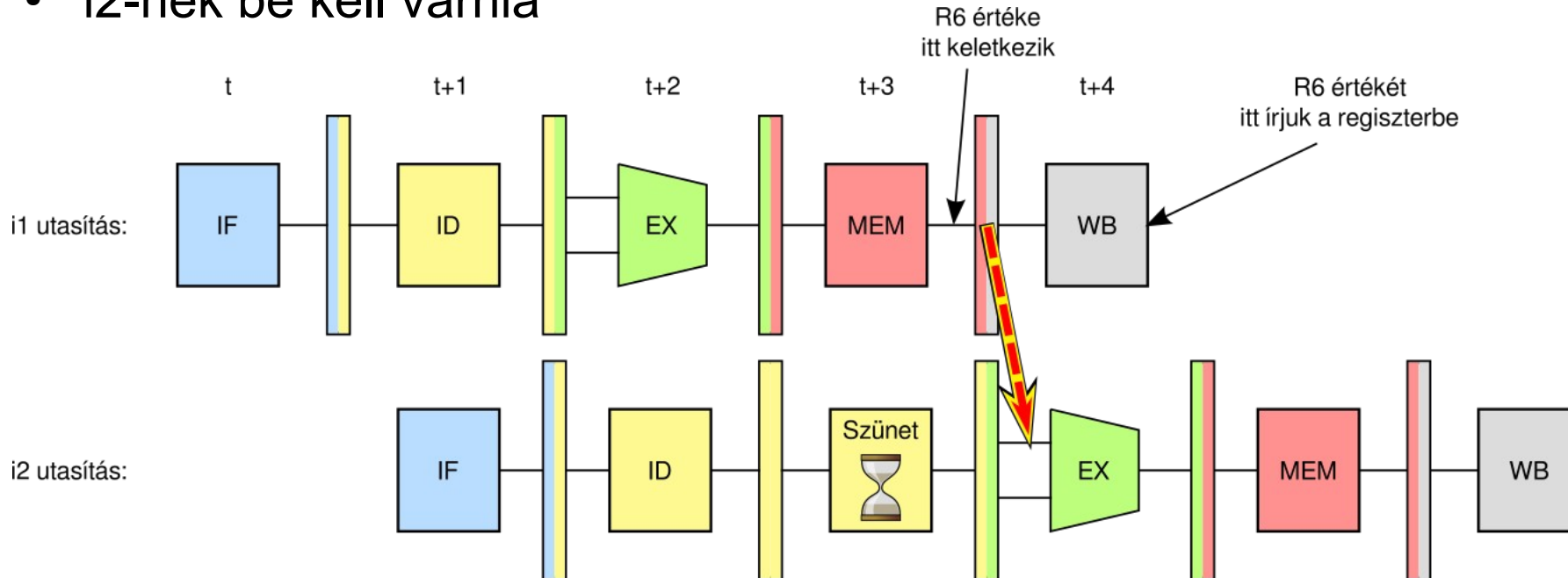


- Forwarding: van, ami ellen nem véd:

i1: R6 ← MEM[R2]

i2: R7 ← R6 + R4

- R6: értéke a MEM végén áll elő
- i2-nek az EX elején kellene
- i2-nek be kell várnia



- Adatfüggőségek:
 - **RAW**: későbbi utasítás olvassa a korábban írt regisztert/címet
 - Megoldás: forwarding/várakozás
 - **WAR**: későbbi utasítás írja a korábban olvasott regisztert/címet
 - Ebben a pipeline-ban nem igényel kezelést
 - **WAW**: több utasítás ugyanazt a regisztert/címet írja
 - Ebben a pipeline-ban nem igényel kezelést
 - **RAR**: több utasítás ugyanazt a regisztert/címet olvassa
 - Sosem igényel kezelést

- Példa:

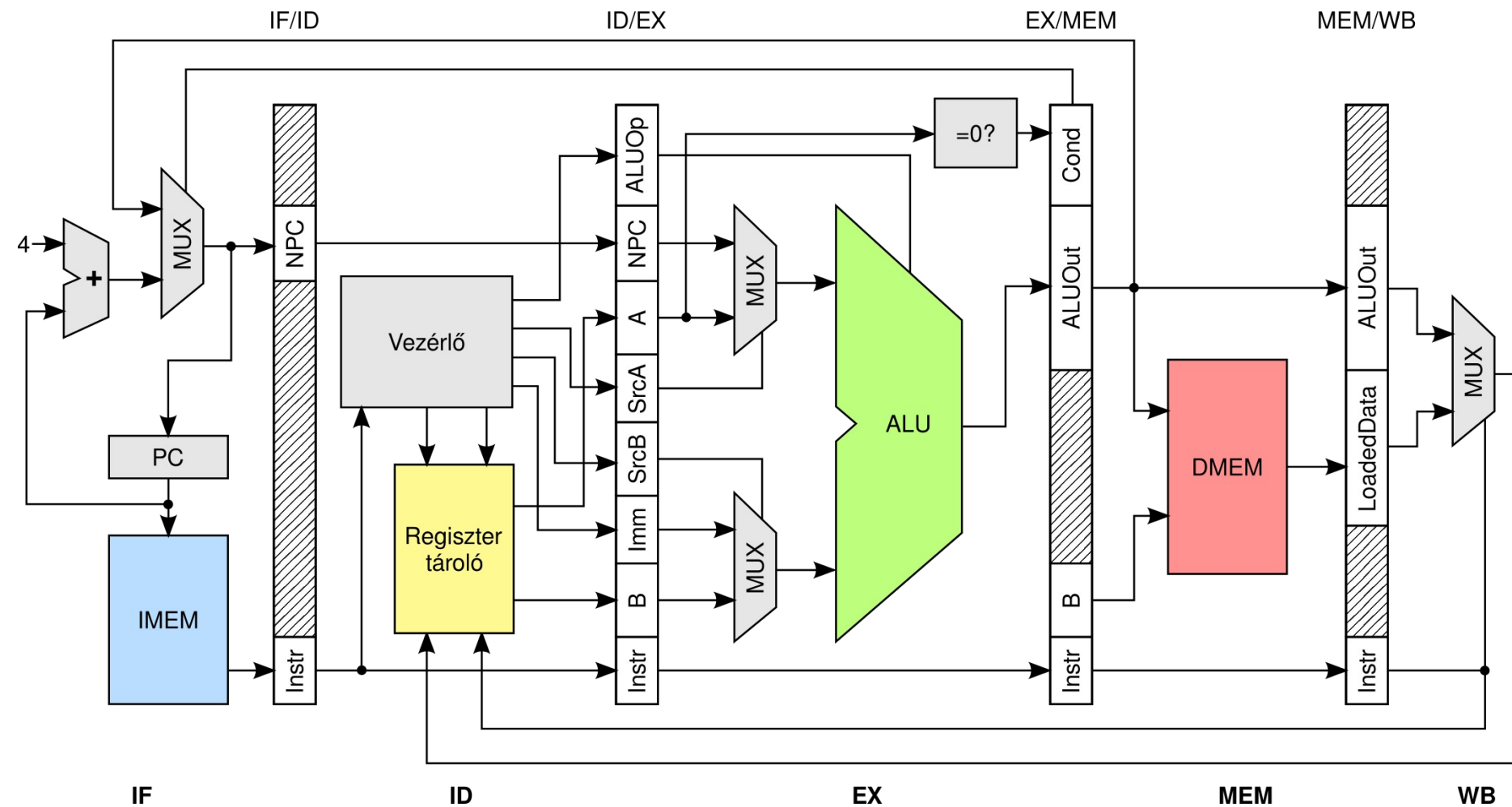
```
i1:  R3 ← MEM[R2]
i2:  R1 ← R2 * R3
i3:  R4 ← R1 + R5
i4:  R5 ← R6 + R7
i5:  R1 ← R8 + R9
```

- WAW: i2 ↔ i5
- WAR: i3 ↔ i4

- Feltételes ugró utasításokkal baj van
 - IF betölti őket
 - Ciklusfeltétel & ugrási cím csak az EX-ben derül ki
 - Addig honnan vegyük a további utasításokat?
- Megoldások:
 - Álljunk meg és várjuk meg, míg az EX-be kerül → Intel 80386
 - Tippeljük meg a kimenetelt és az ugrási címet
→ **elágazásbecslés**

- **Statikus elágazásbecslés** → nem adaptív
 - Ugrás megghiúsulására voksol
 - Ingyen van.
Ha rossz a tipp, érvénytelenítjük a tévesen behozottakat
 - Ugrás bekövetkezésére voksol
 - De akkor honnan veszi a címet?
 - Akkor jó, ha a cím hamarabb előáll, mint a feltétel
 - Visszafele ugrás: megvalósulást, előre: megghiúsulást tippel
 - Motiváció: ciklusok visszafele ugranak
- **Dinamikus elágazásbecslés**
 - Rátanul minden ugrás viselkedésére
 - Drasztikusan jobb, mint a statikus
 - Nem utópia! Sőt, nem is túl bonyolult!

- Mit tanultunk eddig?
 - Pipeline elvet
 - Fázisok pipeline regisztereken keresztül kommunikálnak
 - Hogy hogyan kell kezelni az egymásrahatásokat
- Csináljunk saját pipeline-os processzort!



- ALU: központi szerep

- Számol:

$R1 \leftarrow R1 + 42$

$R1 \leftarrow R2 + R3$

- $PC \leftarrow PC + 42$ (JUMP +42)

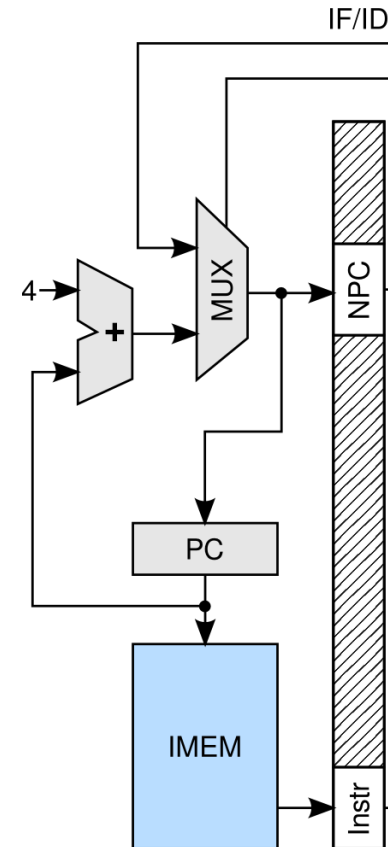
$R1 \leftarrow \text{MEM}[R2 + 42]$

- Első operandus: **regiszter** vagy **PC**
- Második operandus: **regiszter** vagy **konstans** (immediate)
- Feltételt értékel ki:

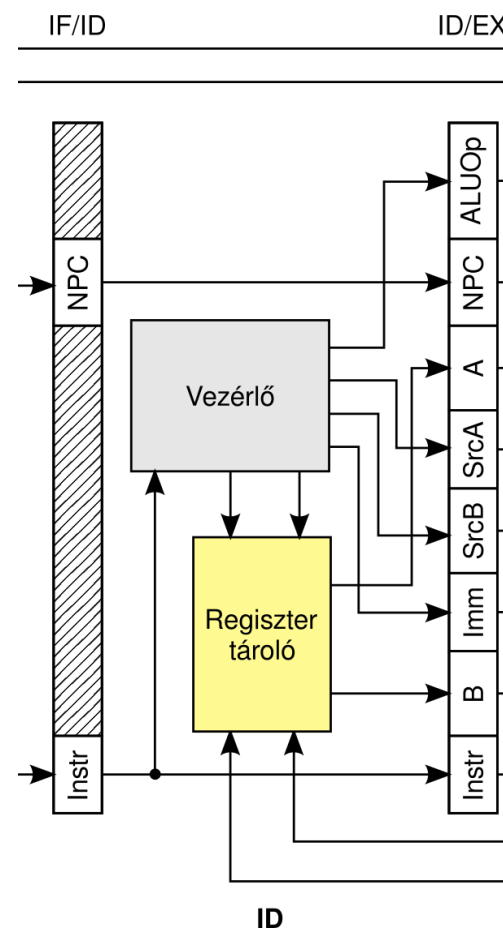
$\text{JUMP } -28 \text{ IF } R1 == 0$

- Az ID feladata:
 - Operandusok előkészítése
 - Operanduskiválasztó jel előállítása
 - Műveleti kód (+, -, *, /)

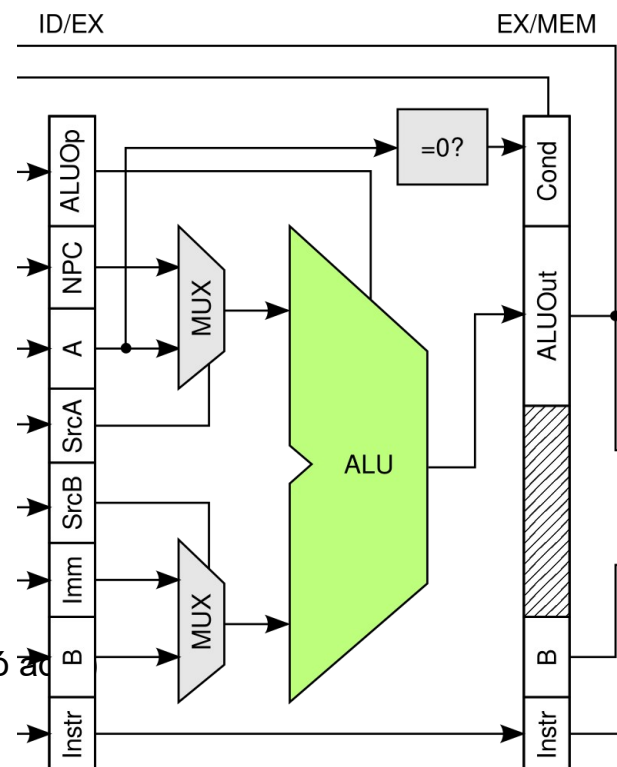
- Utasításszámláló update:
 - Ha ugrás (**EX/MEM.Instr Opcode==branch**),
és ugrani kell (**EX/MEM.Cond==TRUE**):
 - **PC $\leftarrow$ EX.MEM.ALUOut**
 - Egyébként:
 - **PC $\leftarrow$ PC+4**
- Utasításszó és (új) programszámláló továbbadása:
 - **IF/ID.NPC $\leftarrow$ PC**
 - **IF/ID.Instr $\leftarrow$ IMEM[PC]**



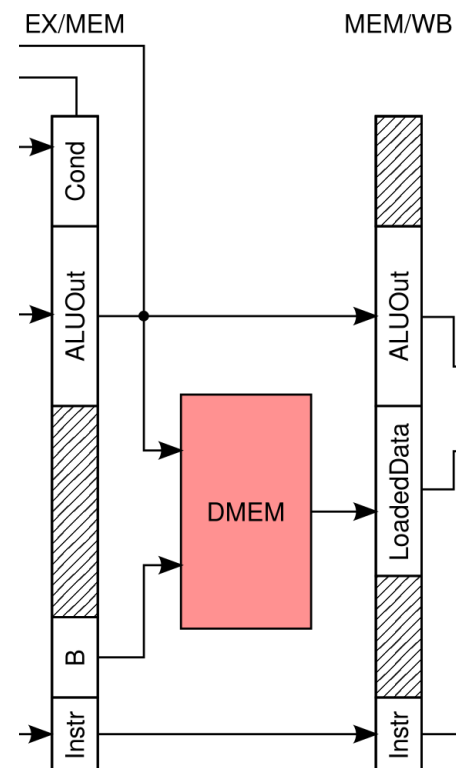
- Operandusok előállítása:
 - Első operandus:
 - $ID/EX.NPC \leftarrow IF/ID.NPC$
 - $ID/EX.A \leftarrow Reg [IF/ID.Instr.ra]$
 - Első operandus kiválasztójel:
 - Ha ugró utasítás ($IF/ID.Instr.Opcode == branch$):
 - $ID/EX.SrcA \leftarrow npc$
 - Egyébként:
 - $ID/EX.SrcA \leftarrow regA$
 - Második operandus:
 - $ID/EX.Imm \leftarrow IF/ID.Instr.imm$
 - $ID/EX.B \leftarrow Reg [IF/ID.Instr.rb]$
 - Második operandus kiválasztójel:
 - Ha konstans ($IF/ID.Instr.HasImm$):
 - $ID/EX.SrcB \leftarrow imm$
 - Egyébként:
 - $ID/EX.SrcB \leftarrow regB$
- Műveleti kód előállítása:
 - Ha aritmetikai az utasítás ($IF/ID.Instr.Opcode == arithm$):
 - $ID/EX.ALUOp \leftarrow IF/ID.Instr.Func$
 - Egyébként (utasításszámláló növelés, címszámítás):
 - $ID/EX.ALUOp \leftarrow „+”$
- Utasításszó továbbadása:
 - $ID/EX.Instr \leftarrow IF/ID.Instr$



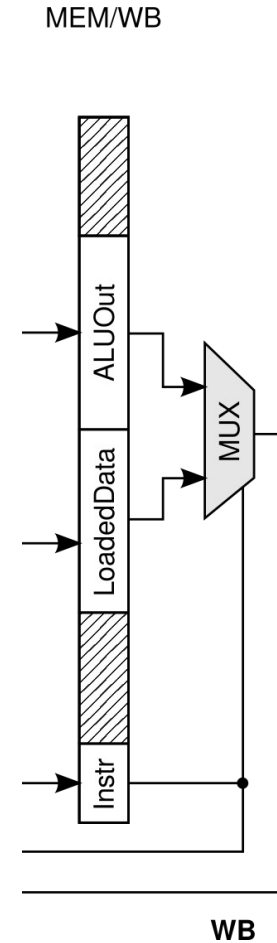
- ALU bemenetei:
 - Ha **ID/EX.SrcA == npc**
 - **ALU.A $\leftarrow$ ID/EX.NPC**
 - Ha **ID/EX.SrcA == regA**
 - **ALU.A $\leftarrow$ ID/EX.A**
 - Ha **ID/EX.SrcB == imm**
 - **ALU.B $\leftarrow$ ID/EX.Imm**
 - Ha **ID/EX.SrcB == regB**
 - **ALU.B $\leftarrow$ ID/EX.B**
 - Művelettípus:
 - **ALU.Op $\leftarrow$ ID/EX.ALUOp**
- Eredmény tárolása:
 - **EX/MEM.ALUOut $\leftarrow$ ALU.Out**
- Összehasonlító egység:
 - **EX/MEM.Cond $\leftarrow$ ID/EX.A == 0**
- Store utasítás esetén B továbbítása
 - **EX/MEM.B $\leftarrow$ ID/EX.B** (Ez lesz majd a beírandó adat)
- Utasításszó továbbadása:
 - **EX/MEM.Instr $\leftarrow$ ID/EX.Instr**



- Memóriacím:
 - **ALUOut**
- Store esetén (**ID/EX.Instr.Opcode == Store**)
 - **MEM[EX/MEM.ALUOut] $\leftarrow$ EX/MEM.B**
(Beírandó adat: B)
- Load esetén (**ID/EX.Instr.Opcode == Load**)
 - **MEM/WB.LoadedData $\leftarrow$ MEM[EX/MEM.ALUOut]**
- Aritmetikai utasítás esetén:
 - **MEM/WB.ALUOut $\leftarrow$ EX/MEM.ALUOut**
(Eredmény továbbadása)
- Utasításszó továbbadása:
 - **MEM/WB.Instr $\leftarrow$ EX/MEM.Instr**

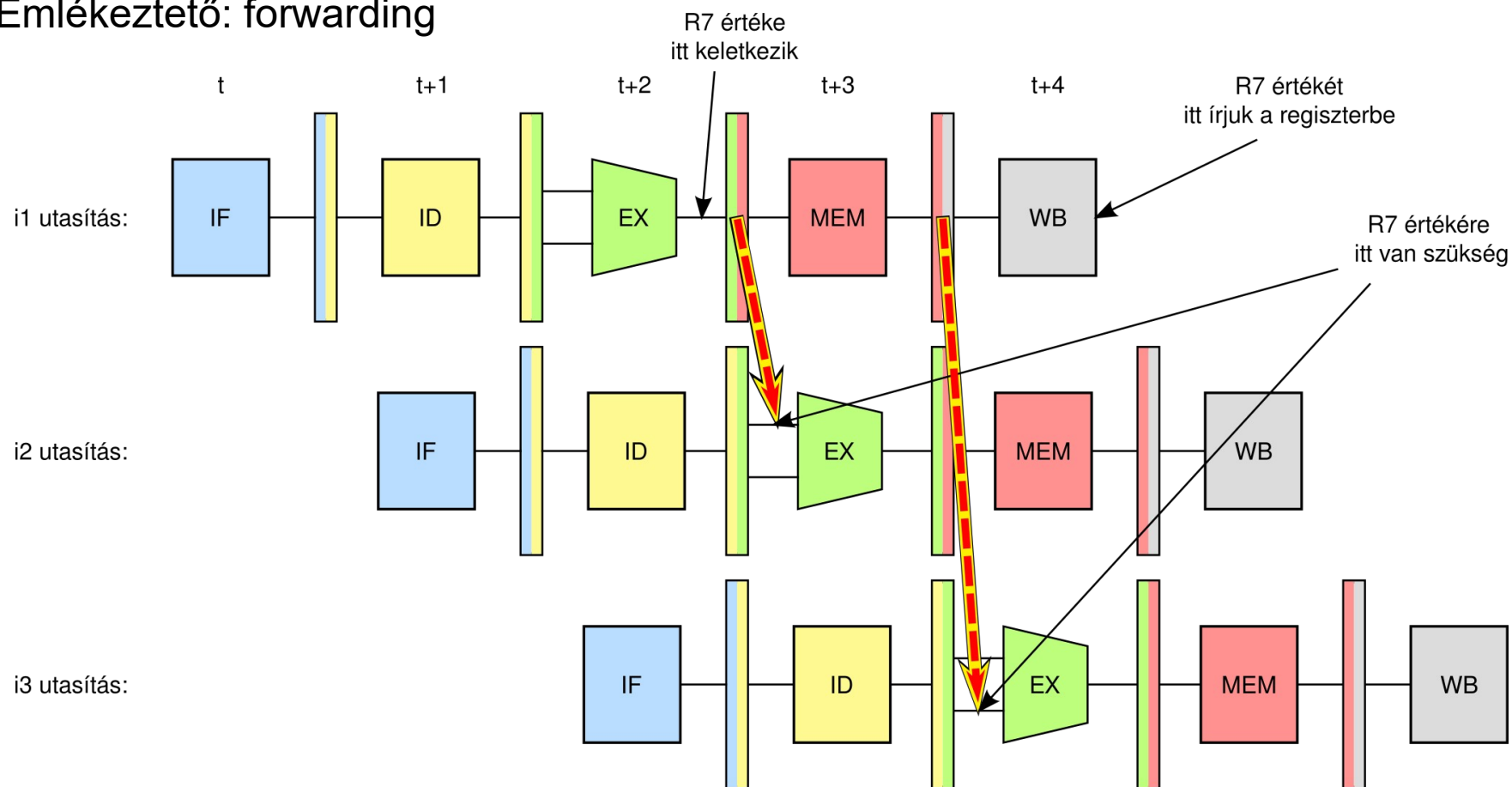


- Regisztertároló (**Reg[]**) frissítése
 - Aritmetikai utasítás esetén
(**MEM/WB.Instr.Opcode == arithm**)
 - **Reg[MEM/WB.Instr.rd] ← MEM/WB.ALUOut**
 - Load utasítás esetén
(**MEM/WB.Instr.Opcode == Load**)
 - **Reg[MEM/WB.Instr.rd] ← MEM/WB.LoadedData**





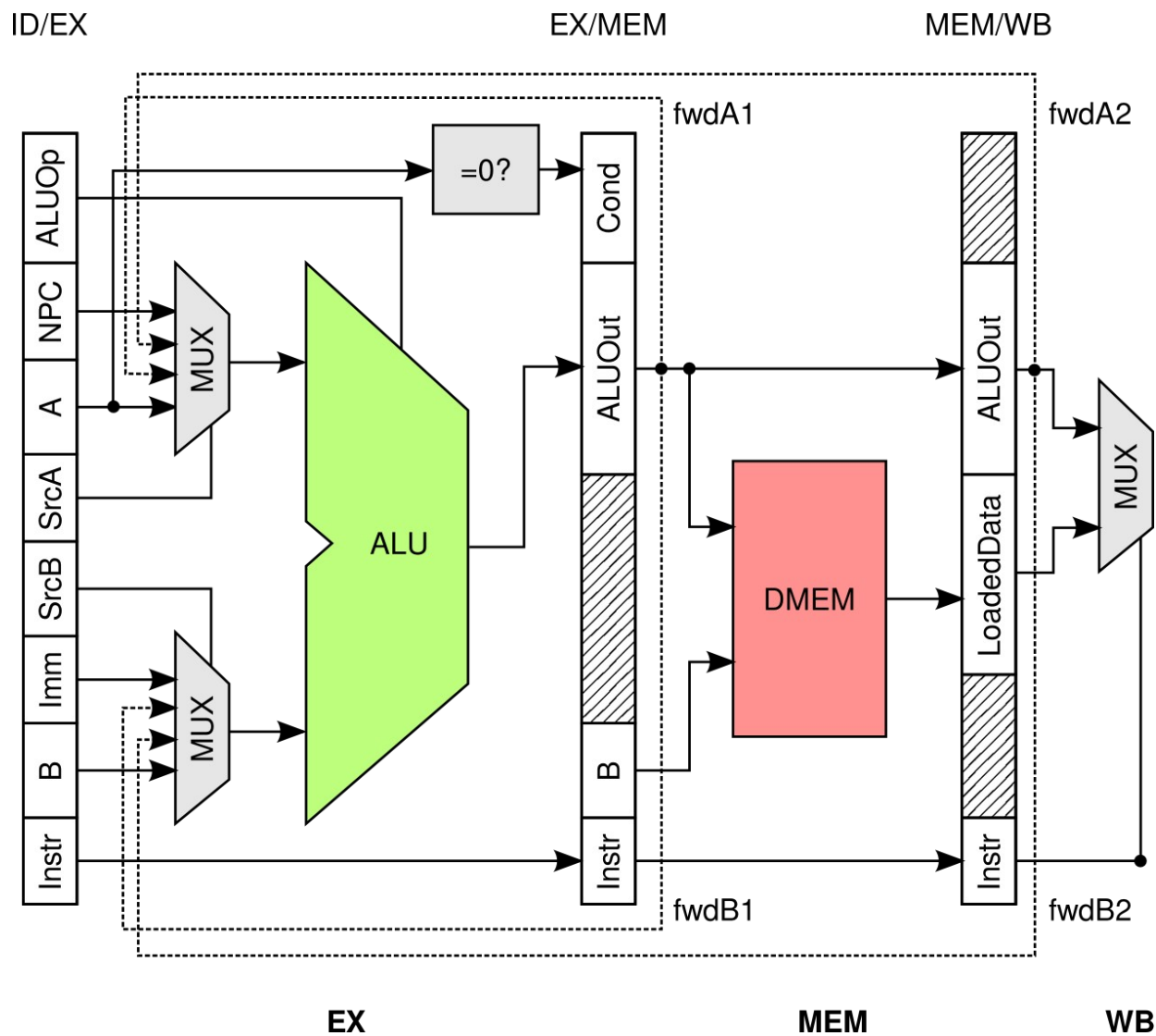
- Emlékeztető: forwarding



i1: R7 ← R1 + R5

i2: R8 ← R7 + R2

i3: R5 ← R8 + R7



- Honnan tudja az ID, hogy mik lesznek az ALU operandusai?
 - Két operandus
 - Mindkettő kétféle lehet (NPC $\leftrightarrow$ regiszter ill. immediate $\leftrightarrow$ regiszter)
 - Regiszterek három helyről származhatnak:
 - ID-től
 - ALU kimenetről
 - Egyel korábbi ALU kimenetről
- Kiválasztó logika (ID-ben!):
 - pl. 1-es operandusra, **ID/EX.SrcA** lehet 0, 1, 2, 3:
 - **npc**: **IF/ID.Instr.Opcode == branch**
 - **fwdA1**: ha **IF/ID.Instr.Opcode==arithm &&**

IF/ID.Instr.ra == ID/EX.Instr.rd
 - **fwdA2**: ha **IF/ID.Instr.Opcode==arithm &&**
IF/ID.Instr.ra == EX/MEM.Instr.rd
 - **regA**: ha **IF/ID.Instr.Opcode==arithm**, egyébként
 - Hasonlóan a 2-es operandusra

- Ha szünetet igényel a RAW egymásrahatás:
 - ID az előbbi módon detektálja
 - Szól az IF-nek, hogy most ne hívjon le új utasítást
- Procedurális egymásrahatás:
 - Megáll & vár taktika: 2 szünet beiktatása, mint előbb
 - „Csak nem következik be” taktika:
 - Folytatja a következő utasítások lehívását
 - Ha mégis ugrás történt, a két félig kész utasítást hatástalanítani kell:
 - Utasítások egy „Valid” flag-et cipelnek magukkal
 - Ha kiderül, hogy félbe kell hagyni őket, Valid=0
 - Valid=0-ás utasítást a MEM és a WB fázis semmibe vesz

- Mit tanultunk eddig?
 - Pipeline elvet
 - Fázisok pipeline regisztereken keresztül kommunikálnak
 - Hogy hogyan kell kezelni az egymásrahatásokat
 - Csináltunk saját pipeline-os processzort
 - A processzorunk már az egymásrahatásokat is kezeli
- Addig jó, amíg valami rendkívüli nem történik
- Mi történhet?
 - Kivétel!
 - Periféria kérés,
 - Laphiba,
 - Védelmi hiba,
 - Érvénytelen utasítás,
 - Stb.

- Kívánatos viselkedés:
 - Ha az i . utasítás közben keletkezik kivétel
 - Tűnjön úgy, mintha $<i$ utasítások befejeződtek volna
 - És a $>i$ utasítások egyáltalán el sem kezdődtek volna
- ***pontos kivételkezelés***
- Pipeline feldolgozás esetén nem triviális
- A kivételkor jó lenne megakadályozni, hogy a későbbi utasításoknak hatása legyen
- ... de lehet, hogy már meg is történt a baj

- Melyik fázisban mi történhet:
 - IF fázis: Laphiba, védelmi hiba
 - ID fázis: Érvénytelen utasítás
 - EX fázis: Aritmetikai hiba (pl. integer túlcsordulás)
 - MEM fázis: Laphiba, védelmi hiba
 - WB fázis: Itt nem történhet kivétel

	1	2	3	4	5	6
$R_k \leftarrow R_m + R_n$	IF	ID	EX	MEM	WB	
$R_i \leftarrow \text{MEM}[R_j]$		IF	ID	EX	MEM	WB

- 1. utasítás: EX fázisban túlcsordulás
- 2. utasítás: IF fázisban laphiba
→ Felborul a sorrend! Egy későbbi utasítás hibája jön előbb!!

- Egy lehetséges megoldás:
 - A kivételt nem kezeljük azonnal
 - Csak feljegyezzük, hogy volt (utasítás görgeti maga előtt)
 - Tényleges kezelés: WB fázisban, vagy utána
→ sorrend garantáltan helyes marad!

	1	2	3	4	5	6
$R_k \leftarrow R_m + R_n$	IF	ID	EX	MEM	WB	
$R_i \leftarrow \text{MEM}[R_j]$		IF	ID	EX	MEM	WB

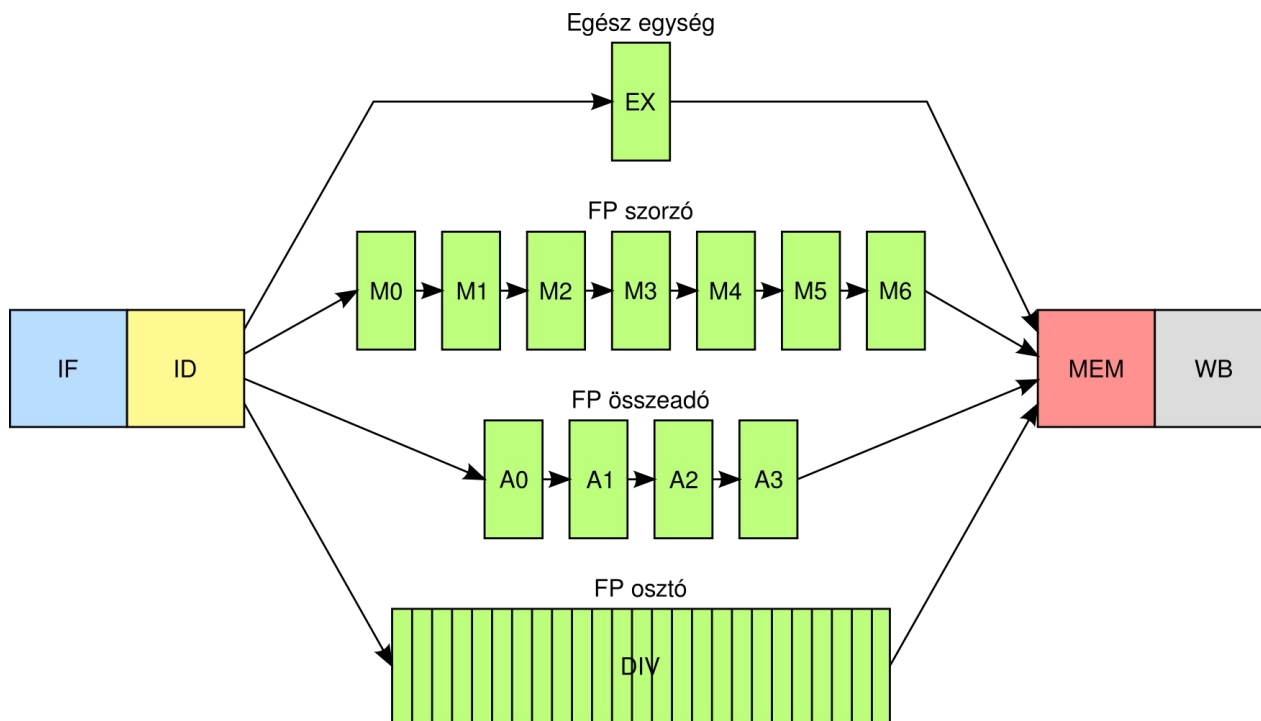
- Áttekintettük, mi a helyzet a kivételekkel
- Újabb csavar: mi van, ha az EX fázis nem mindenkinek egyforma hosszú?

- Aritmetikai műveletek **késleltetése**:
 - Lebegőpontos műveletek tovább tartanak
 - Szorzás tovább tart, mint az összeadás
 - Osztás még tovább tart
- A műveleti egységek lehetnek maguk is pipeline szervezésűek
 - Pl. az összeadás 3 ciklus késleltetésű,
 - ...de minden ciklusban új összeadást tud kezdeni

→ **Iterációs idő** = 1

ELTÉRŐ IDEJŰ ARITMETIKAI MŰVELETEK

Egység	Késleltetés	Iterációs idő
Integer ALU	1	1
FP összeadó	4	1
FP szorzó	7	1
FP osztó	25	25



- Érdekesség:
 - Végrehajtás kezdete: *in-order*, vége: *out-of-order* is lehet!

D4 ← D1 * D5	IF	ID	M0	M1	M2	M3	M4	M5	M6	MEM	WB
D2 ← D1 + D3	IF ID A0 A1 A2 A3 MEM WB										
D0 ← MEM [R0+4]											
MEM [R0+8] ← D5				IF	ID	EX	MEM	WB			

- Nem volt gond, utasítások függetlenek
- Nem lesz mindig így
- ID egység új feladatai:
 - a program szemantikájának megőrzése
 - újfajta egymásrahatások kezelése

ÚJFAJTA FELDOLGOZÁSI EGYMÁSRAHATÁSOK

Utasítás	1	2	3	4	5	6	7	8	9	10	11
D4 ← D1 * D5	IF	ID	M0	M1	M2	M3	M4	M5	M6	MEM	WB
...		IF	ID	EX	MEM	WB					
...			IF	ID	EX	MEM	WB				
D2 ← D1 + D3				IF	ID	A0	A1	A2	A3	MEM	WB
...					IF	ID	EX	MEM	WB		
...						IF	ID	EX	MEM	WB	
D0 ← MEM [R0+4]							IF	ID	EX	MEM	WB

- Gond:
 - 10.: egyszerre hárman MEM-ben
 - 11.: egyszerre hárman WB-ben
- Megoldás:
 - ID tudja, mi mennyi ideig tart → előre látja ezt a helyzetet
 - Ha valaki feldolgozási egymásrahatást okozna → parkoltatja

- Adatfüggőségek még nagyobb gondot okoznak
- Oka: hosszabb pipeline:
 - Későn áll elő az eredmény
→ ha valaki a korábbi eredményétől függ, többet kell várnia
 - Több utasítás van feldolgozás alatt
→ nagyobb a sansz a RAW függőségre

Utasítás	1	2	3	4	5	6	7	8	9	10	11	12	14	14	15	16	17
D4 ← MEM [R0+4]	IF	ID	EX	ME	WB												
D0 ← D4 * D6		IF	ID	SZ	M0	M1	M2	M3	M4	M5	M6	ME	WB				
D2 ← D0 + D8			IF	SZ	ID	SZ	SZ	SZ	SZ	SZ	SZ	A0	A1	A2	A3	ME	WB
MEM [R0+4] ← D2					IF	SZ	SZ	SZ	SZ	SZ	SZ	ID	EX	SZ	SZ	SZ	ME

- WAW függőség:
 - Két utasítás ugyanabba a regiszterbe menti az eredményét
- Ha megengedjük, hogy az utasítások sorrenden kívül fejeződjenek be → baj lehet!
 - Mert lehet, hogy a végén a korábbi eredménye kerül bele!
 - Az ID fázisnak detektálnia, és szünetekkel kezelnie kell

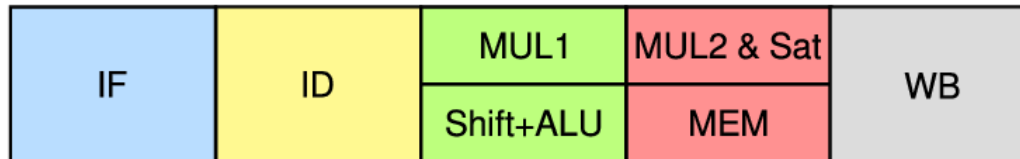
Utasítás	1	2	3	4	5	6	7	8
$D2 \leftarrow D1 + D3$	IF	ID	A0	A1	A2	A3	MEM	WB
...		IF	ID	EX	MEM	WB		
$D2 \leftarrow \text{MEM}[R0+4]$			IF	ID	EX	MEM	WB	

- Láthatóan rossz.
 - Megoldás: ID 2 szünetet iktat be a 3. utasításnál



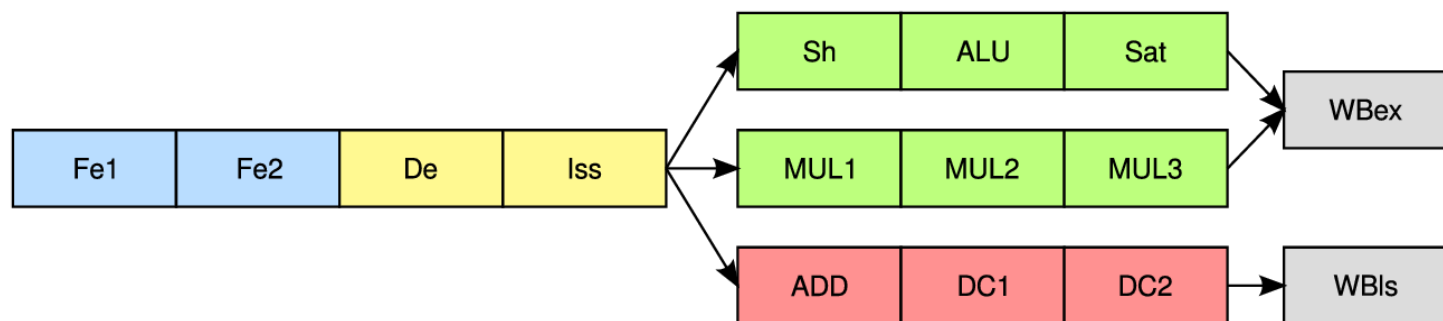
Alternatív pipeline struktúrák

- 5 fokozatú, hasonlít a tanultakhoz
- Eltérés:
 - Szorzás két fázist igényel → második lépése: MEM
 - Van telítődő aritmetika → telítődés a MEM-ben



AZ ARM1176JZF-S PROCESSZOR PIPELINE-JA

- A Raspberry Pi processzora
- Mélység: 8
 - IF: 2 fázis (Fe1: lehívás, Fe2: elágazásbecslés)
 - ID: 2 fázis (De: dekódolás, Iss: regisztertároló kiolvasása)
- 3 műveleti egység, késleltetés: 3, iterációs idő: 1
 - ALU (Sh: shift, ALU: számolás, Sat: telítődés)
 - Szorzás (MUL1, MUL2, MUL3)
 - Memóriaműveletek (ADD: címszámolás, DC1, DC2: adatcache elérés)
 - Az ALU és a szorzó nem megy egyszerre, a többi igen





HÁLÓZATI RENDSZEREK
ÉS SZOLGÁLTATÁSOK
TANSZÉK

